

PDFVIEW

A PDF reader that borrows both of its renderers from software already on the machine, and spends its own code on the two things nobody else will do for it: the window, and getting along with Explorer.

VERIFIED 2026-09-18

WARM START 915 MS

2 FLOOR COSTS, 433 MS

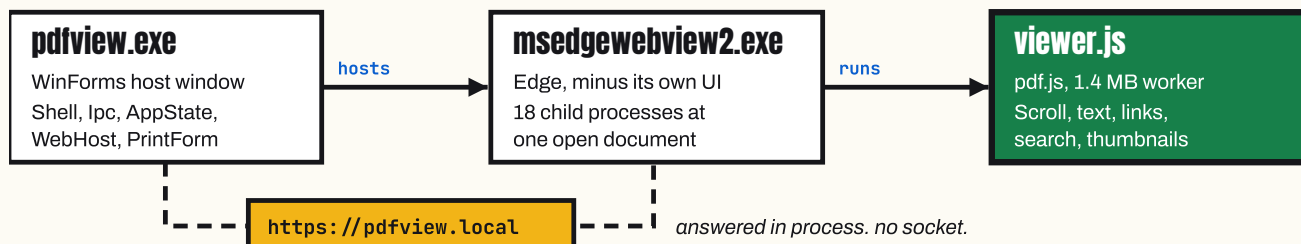
1 GAP OPEN

Every number on this page was read off one machine on **2026-09-18**, opening `docs/sample.pdf`, a six-page document. Nothing here is an intention. Where something is intended and not built it carries a gap mark and says so.

THE SHAPE OF IT

Three processes, two PDF renderers, and one of them is not even in the same process tree as the app. Drawn because a paragraph describing how they connect is a diagram that has not been drawn yet.

THE WINDOW



THE ICON



The two halves never speak. The app can be closed, uninstalled or crashing and Explorer still draws page previews, because the thumbnail handler is a COM server Windows loads into its own sandbox on demand. That separation is the reason the thumbnail half gets a different PDF renderer, and it is the first real design choice on the list.

CHOICES THAT DECIDED THE REST

Four decisions, each of which closed off a set of other decisions. Everything downstream is a consequence rather than a preference.

NEITHER RENDERER IS OURS

The window runs **pdf.js inside WebView2**. The thumbnail handler runs **Windows.Data.Pdf**, the rasteriser built into Windows 10 and 11. Two different PDF engines in one product looks like an accident and is not.

The handler is loaded into a process Windows owns, alongside every other thumbnail provider on the machine, with a hard time budget and no UI thread of its own. Putting a 1.4 MB JavaScript engine and a browser process in there to draw a 96-pixel square would be indefensible. Windows.Data.Pdf is already resident, takes a stream, and hands back a bitmap. It is the right tool in that seat and the wrong one in the window, where selectable text, links, outlines and whole-document search are the entire point and Windows.Data.Pdf offers none of them.

VERIFIED 2026-09-18

Both paths were driven directly, not inferred. `Renderer.RenderFile` called through reflection on the shipped `PdfThumb.dll` returns a 198×256 bitmap for `sample.pdf` at `cx=256`. The same file through `IShellItemImageFactory::GetImage` with `SIIGBF_THUMBNAILOONLY` returns the identical image, which is how we know the shell is handing back exactly what the handler drew.

NOTHING LISTENS ON A SOCKET

The viewer is a web page, so it needs an origin. It does not get a local web server.

`WebHost` registers a `WebResourceRequested` filter for `https://pdfview.local/*` and answers every request in process: static files out of `app/` and `vendor/`, the document itself through `/api/file` with real HTTP range support, and a handful of endpoints that put up native dialogs.

The payoff is that no other program on the machine can reach any of it. There is no port to find, no loopback binding for a firewall to ask about, and no window where a different process could fetch the document you have open. The cost is that a modal dialog cannot run inside the request handler without wedging the browser process, so `/api/open` and `/api/save` take a deferral and answer once the dialog closes.

ONE PROCESS, MANY WINDOWS

A second launch takes a named mutex, finds it held, writes its file path down a named pipe to the first instance and exits. Every document therefore lives in one process and one taskbar group. If the mutex holder is gone or wedged the second launch carries on and opens its own window rather than refusing to start.

HKCU ONLY, AND NO SURROGATE OF OUR OWN

Registration writes to `HKEY_CURRENT_USER` and nowhere else, so installing needs no administrator and uninstalling is a subtree delete. The thumbnail CLSID gets an `InprocServer32` and no `AppID`, which is deliberate: Windows already runs stream-based thumbnail handlers in its own isolated low-privilege host, and declaring a `DllSurrogate` would opt out of that in favour of a plain `dllhost`.

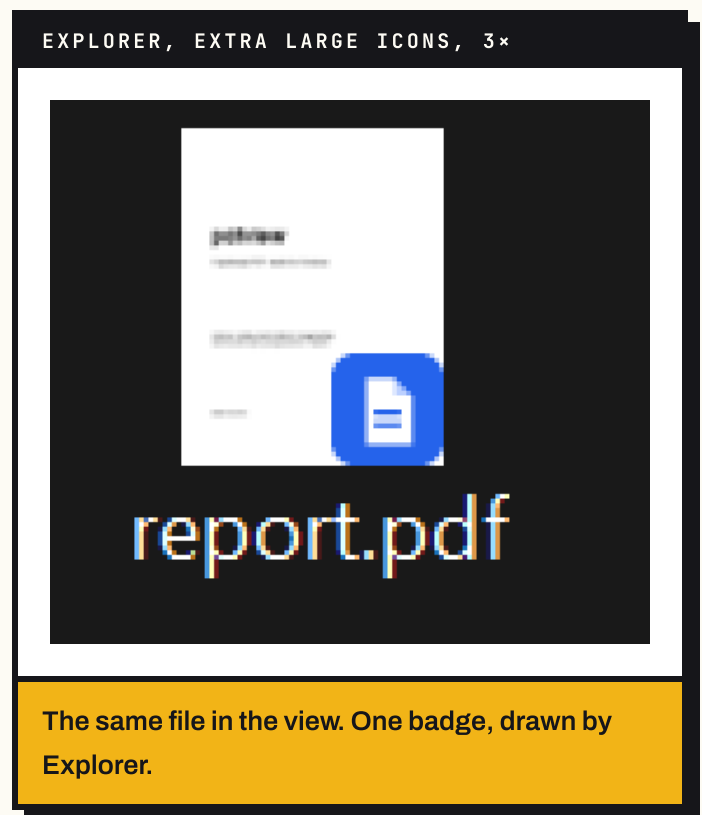
DECISION	WHAT IT BOUGHT	WHAT IT COST	STATE
pdf.js in WebView2 for the window	Selectable text, links, outline, whole-document search	A browser process, and 433 ms of the launch	✅ Works
Windows.Data.Pdf for thumbnails	A handler small enough to live in the shell's sandbox	Two PDF engines to reason about	✅ Works
In-process origin, no socket	Nothing else on the machine can reach the document	Dialogs need deferrals	✅ Works
One process, many windows	One taskbar group, shared browser process	A wedged first instance is a visible failure mode	⚠️ Surprises
Self-contained publish	No .NET install for the user	148 MB, of which 50.5 MB is never loaded	❌ Gap

WINDOWS IS THE OTHER USER

The person is one user of this program. The shell is the other, and it is the one that judges the app before anybody opens it: in a folder listing, in Alt+Tab, in the taskbar preview. Most of the work in this section is about being legible to something that is not looking very hard.

THE ICON THAT ARRIVED TWICE

Explorer composites the file association icon over a document thumbnail **at paint time, in the view**. Not in the thumbnail cache, and not in the handler's output. That distinction is the whole bug: the handler had been drawing the pdfview mark into the bottom-right corner at 34% of the thumbnail's width, and Explorer was then drawing its own badge on top of it.



So the fix was to draw nothing. `DrawBadge`, `LoadBadge`, the three tuning constants and the embedded `.ico` resource all came out of `PdfThumb`, and the corner is left to the

shell, which knows the right size for it at every DPI and every icon view.

WATCH OUT

The thumbnail cache keys on path plus modification time, so a corrected handler is invisible until the cache lets go. Nothing you do to the code will show up in a folder you have already looked at.

Fastest check: copy a PDF to a new filename. Proper clear: stop Explorer, delete `%LOCALAPPDATA%\Microsoft\Windows\Explorer\thumbcache_*.db`, start it again.

There is a second cache in front of that one, the COM surrogate, which keeps `PdfThumb.dll` loaded and locked once it has drawn anything; a rebuild fails with *file in use* until it is killed.

SIX HUNDRED MILLISECONDS OF EMPTY WINDOW

The window is on screen at 296 ms. The first page of the document is on it at 865 ms. For the **569 ms** in between there is a real, sized, focusable, Alt+Tab-able window with no content in it at all, and there is no honest way to avoid that: WebView2 cannot start without a window to attach to.

What the app controls is what that window is painted with. The background colour the viewer settled on is posted back to the host and written to `state.json`, and the next launch paints the form and the WebView2's own default background that colour before anything loads. A dark-theme reader never sees a white rectangle, and the taskbar preview and Alt+Tab thumbnail show a plausible window rather than a flash. It is one string in a settings file doing the work of a splash screen.

THE SHELL OWES YOU ONE

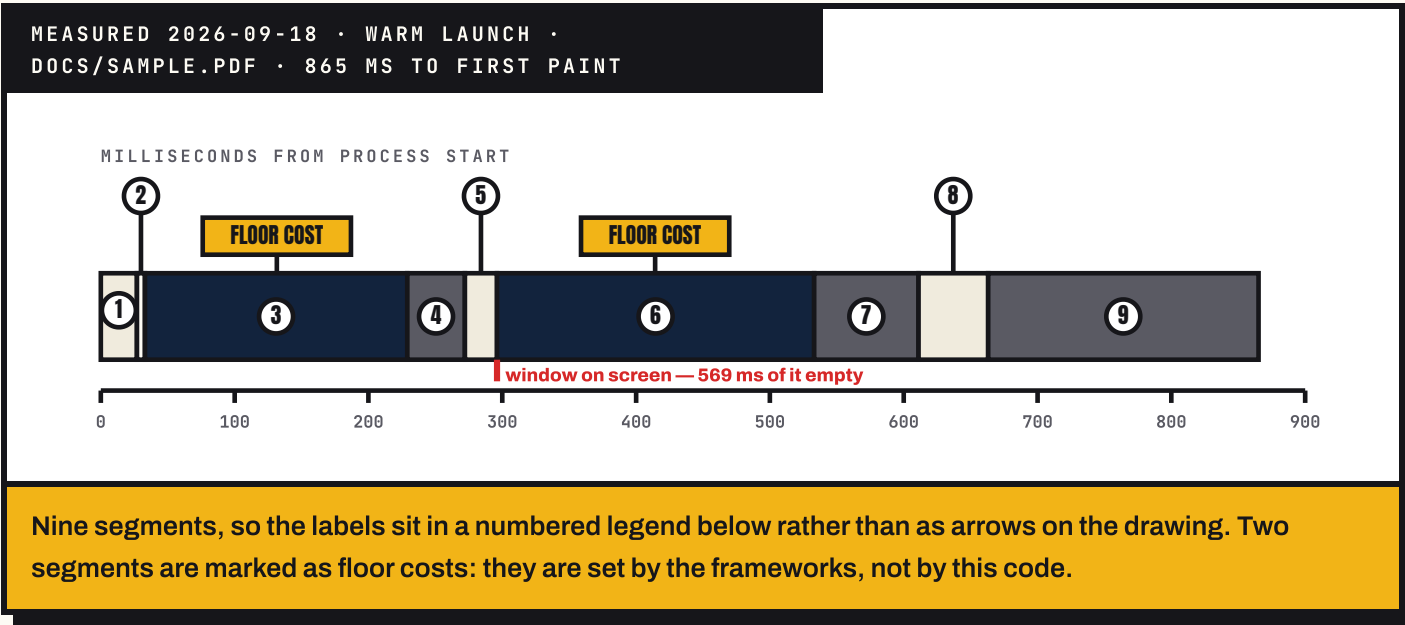
Because thumbnail handlers run in a host Windows provides, a fault in `PdfThumb` cannot take Explorer down. That is worth more than it sounds: it means the rendering path can be aggressive about odd files, and the failure mode of a bad PDF is one icon falling back to the generic document glyph.

PLACEMENT, AND MONITORS THAT WENT AWAY

Window bounds are remembered and restored, each additional window cascades 28 pixels so they do not stack exactly, and the restored rectangle is tested against every attached screen. A window saved on a monitor that is no longer there gets dragged back to the primary work area rather than opening somewhere nobody can reach it.

WALKTHROUGH: ONE LAUNCH

A representative warm launch of the current build, start to first page on screen. Numbered because this genuinely is a sequence, and every step below waits for the one above it.



1	Host and .NET runtime start	27 ms	2	Visual styles and per-monitor DPI	6 ms
3	The first <code>new Form()</code>	196 ms	4	WebView2 control, icon, placement	43 ms
5	Window shown	24 ms	6	Browser process spawn	237 ms
7	HTML, CSS, viewer.js, pdf.js	78 ms	8	pdf.js opens the document	52 ms
9	First page drawn	202 ms			

3

THE FIRST FORM IS THE EXPENSIVE ONE

196 ms

Not `EnableVisualStyles`, which is 2 ms, and not the window handle, which is 9 ms. Constructing the very first WinForms `Form` in the process costs 130 to 250 ms of type loading and static initialisation across `System.Windows.Forms` and `System.Drawing`, GDI+ startup included. The second `Form` in the same process costs nothing measurable.

This is why the window is now created before the IPC listener and the hidden marshalling form instead of after them. Whichever `Form` is first pays this, so it may as well be the one the user is waiting for; everything else moved behind it, into the window where the browser is spawning anyway.

6

THE BROWSER SPAWN IS PER PROCESS, NOT PER WINDOW

237 ms

Measured both ways. The first window's `EnsureCoreWebView2Async` costs 183 to 263 ms. A second window in the same process, opened by handing a different file to the running instance, costs **39 ms**, and its `WebView2` control constructs in 0.2 ms rather than 33 ms.

So roughly 220 ms of that first cost is the browser process starting, paid once. It is also the reason the single-instance rule earns its keep beyond tidiness: the second document you open is about 500 ms faster than the first.

WHAT CHANGED

Measured as 18 interleaved pairs, alternating builds run by run so drift in machine load lands on both. Warm launches, first run of each set discarded.

TO FIRST PAGE ON SCREEN	1.0.0	CURRENT	CHANGE
Median	988 ms	915 ms	-73 ms
25th percentile	907 ms	857 ms	-50 ms
75th percentile	1002 ms	983 ms	-19 ms
Fastest observed	868 ms	818 ms	-50 ms

Seven per cent. It is worth being blunt about why it is not more: two thirds of what remains is the 196 ms first `Form` and the 237 ms browser spawn, and both are floor costs of choosing WinForms and WebView2. The rest of the budget was already fairly tight.

THE FIVE CHANGES, AND WHAT EACH WAS WORTH

CHANGE	MECHANISM	STATE
Compile ahead of time <code>PublishReadyToRun</code> , non-composite	The largest single item. Cuts time-to- <code>Main</code> from ~100 ms to ~27 ms and takes a bite out of the WinForms first-touch cost.	✓ Kept
Window before plumbing <code>Shell.Run</code> reordered	The IPC listener and the hidden marshalling form now start after the window, inside the gap where the browser is spawning.	✓ Kept
Less Edge to start eight added browser switches	No first-run greeting, no extensions, no sync, no component update, no background networking, no crash reporter.	✓ Kept
pdf.js alongside viewer.js two <code>modulepreload</code> links	<code>viewer.js</code> imports pdf.js at its top, so the browser could not see the second fetch until the first had arrived and parsed. Declared in the head they go out together.	✓ Kept
Read the state file once <code>AppState</code> caches, revalidates on mtime	Opening a window used to parse <code>state.json</code> three times: remembered background, remembered placement, recent list. Small on a warm disk, and it removed a real stall (below).	✓ Kept

TWO CORRECTNESS FIXES THAT CAME OUT OF THE SAME PASS

The recent list no longer waits on unreachable drives. It pruned entries by calling `File.Exists` on every remembered path, up to twenty-five of them, on the path that opens a document. One entry on a disconnected share answers in its own time, and the open waited. Paths on UNC roots and drives that are not ready fixed local disks are now taken on trust instead of stated.

The recent list no longer blocks the open. Startup used to `await loadRecents()` before looking at the requested file. The fetch now runs alongside opening the document and is awaited at the one point that genuinely needs it, which is working out what page you left this file on.

PATHWAYS NOT TAKEN

Two optimisations were built, measured and removed. They are written down with their numbers because the next person to have the idea should get to skip the implementation.

COMPOSITE READYTORUN

Linking the whole framework into one ahead-of-time image is the faster of the two R2R modes on a warm start, by a few per cent. It also produces an **87 MB** `pdfview.r2r.dll`, and the first launch after install then waits for the virus scanner to read all of it: **6187 ms against 2141 ms** for the non-composite build, measured as the first run after a fresh publish.

The first launch is the one a new user judges the program on. Non-composite kept.

SPAWNING THE BROWSER DURING WINFORMS START-UP

The two big floor costs are roughly the same length and run one after the other, which is an obvious invitation. The browser process only needs a window handle, and a bare Win32 window costs about a millisecond and touches none of WinForms — a hidden WinForms form would be no use, because it queues behind the same static constructors the UI thread is already inside.

So: a `0x0 WS_POPUP` window on its own STA thread with its own message pump, a throwaway `CoreWebView2Controller` on it, and the real window finds the browser already running. It works. `EnsureCoreWebView2Async` on the real window drops from 183–263 ms to **10–27 ms**.

And the launch is barely faster. Over 40 interleaved runs the median moved from 864 ms to 820 ms, about five per cent, because **the spawn is not free of the thing it is overlapping**: run concurrently with WinForms loading, the same browser start-up stretches from 237 ms to between 400 and 880 ms of wall clock. The two costs contend for the same disk and the same cores. Moving work off the serial path does not help much when the work gets slower by the same amount.

THE TRAP IN THIS ONE

A WebView2 environment is an apartment-bound COM object. The first attempt borrowed the main environment from the warm-up thread and failed with *Unable to cast to ICoreWebView2Environment*, because it had resolved on a thread pool

thread. The working shape is a second environment created on the warm-up thread, naming the identical user data folder and identical browser arguments so the two share one browser process.

Then the two environments open that folder at the same time and contend, which cost more than the warm-up saved until the main one was deferred.

About 180 lines of Win32 interop, a raw window class, a hand-rolled message loop and an extra WebView2 controller, for five per cent that does not clear the noise. Removed.

STILL OPEN

GAP 1 **50.5 MB of WPF ships and is never loaded.** A self-contained publish copies the whole Microsoft.WindowsDesktop.App runtime pack, and that pack carries WPF alongside WinForms. 38 files, headed by `PresentationFramework.dll` at 15.4 MB, `PresentationCore.dll` at 8.1 MB and `System.Windows.Forms.Design.dll` at 5.3 MB, out of 148 MB at the root of `dist`. The app references `Microsoft.Web.WebView2.WinForms` and never touches the WPF flavour.

Removing them is an MSBuild target over `ResolvedFileToPublish`, not a code change. It is left open rather than done quietly, because it changes the shape of a shipped build and the failure mode of getting the list wrong is an app that will not start on somebody else's machine.

PATHWAY	MEASURED	VERDICT
Composite ReadyToRun	6187 ms first launch, vs 2141	 Rejected
Browser warm-up on a bare HWND	864 → 820 ms, n=40	 Rejected
Strip the WPF runtime from publish	-50.5 MB of 148	 Open
Replace the WinForms host with Win32	up to -196 ms, not attempted	 Open

TWO THIRDS OF A COLD START IS TWO FRAMEWORKS WAKING UP.

196 ms for the first WinForms Form and 237 ms for the browser process, out of 915. Neither is this program's code, and neither will move without changing what the program is built on.

MEASURE IT YOURSELF

The instrument is in the shipped build. It is off unless asked for, and when it is off every mark is a single bool test, which is what keeps it on the real start-up path instead of in a build nobody runs.

```
set PDFVIEW_TRACE=1
dist\pdfview.exe docs\sample.pdf

type %LOCALAPPDATA%\pdfview\trace.log
```

Time zero is when Windows created the process, taken from `Process.StartTime`, not when `Main` ran. The host and runtime start-up in between is a real part of what the user waits for and it is the part that is easiest to forget to measure. The page posts its own marks back through the same channel the viewer already uses to talk to the host, so a single timeline covers both sides of the process boundary:

```
27.3 ms (+ 27.3) main
30.1 ms (+ 2.0) visual styles
33.1 ms (+ 3.0) dpi mode
228.7 ms (+ 195.6) viewer form ctor
263.9 ms (+ 35.2) viewer form built
295.8 ms (+ 23.3) window shown
533.5 ms (+ 237.7) corewebview2 ready
610.6 ms (+ 77.1) page: module evaluated
662.9 ms (+ 52.3) page: pdf parsed
865.1 ms (+ 202.2) page: first-paint
```

One further switch exists for bisecting: `PDFVIEW_WEB` points the host at a different `app/` directory, so the front end can be edited without republishing.

pdfview · internals and design record no. 1 · drawn up 2026-09-18

Every figure measured on one Windows 11 machine against `docs/sample.pdf`, six pages, on 2026-09-18. Launch timings are warm starts to first page paint, collected as interleaved pairs so machine drift lands on both builds; cold-start numbers are quoted only where the first-run effect is the point being made, and are not reproducible run to run.

House style: 1993 strategy guide, from `design/HOUSE_STYLE.md`. Verdict marks from `design/SYMBOL_STANDARDS.md` – circle works, triangle will surprise you, square is missing; shape before colour, so the page survives greyscale. Single theme on purpose.